Module 1 · Fondations et environnement de production

Un batch exploitable : traces, verrous, reprise, sur PostgreSQL

ESSENTIEL

Durée visée : 46 min

- 1.1 Pourquoi un batch de production ?
- 1.2 Le vocabulaire en un schéma
- 1.3 Le projet : Boot 4, Batch 6, PostgreSQL
- 1.4 Votre premier job et son lancement
- 1.5 Sous le capot : tables, versions, isolation
- 1.6 Top 10 des pièges

Le batch : lire, traiter, écrire



Batch processing = répétition de traitements sur de grands ensembles de données, sans intervention humaine.

Données accumulées pendant une période : une semaine, un mois, une année.

Les besoins techniques d'un traitement de masse

Besoin	Ce que cela veut dire
Sans intervention humaine	pas d'IHM : tout est automatique, l'humain déclenche au plus
Efficacité	traitements lourds et longs (minutes, heures) : à optimiser
Supervision	savoir ce qui est fait, ce qui reste, et si tout s'est bien passé
Fiabilité	reprendre un lot en échec : une transaction par lot
Pas un scheduler	un planificateur (Quartz, cron, Kubernetes) peut le lancer
Parallélisme	beaucoup de jobs en parallèle pour de gros volumes

Cas d'usage métier

Cas	Exemple
Salaires	fin de mois : calculer et virer les salaires des employés
Factures	un opérateur télécom en génère des millions, en PDF
Bulletins de paie	traitement de masse à chaque fin de mois
E-mails	envois automatiques à partir de fichiers de données
Rapports	quotidiens, hebdomadaires, mensuels
Workflows	exécution automatique sans intervention humaine

Cas d'usage techniques

Cas	Exemple
Tests planifiés	exécuter les tests unitaires à intervalle régulier
Base de données	mises à jour automatiques à heure fixe (par exemple 2 h du matin)
Files d'attente	consommer les messages d'un broker puis les traiter
Intégrité	contrôler l'intégrité d'un serveur d'applications
Données factices	remplir une base pour un test de charge

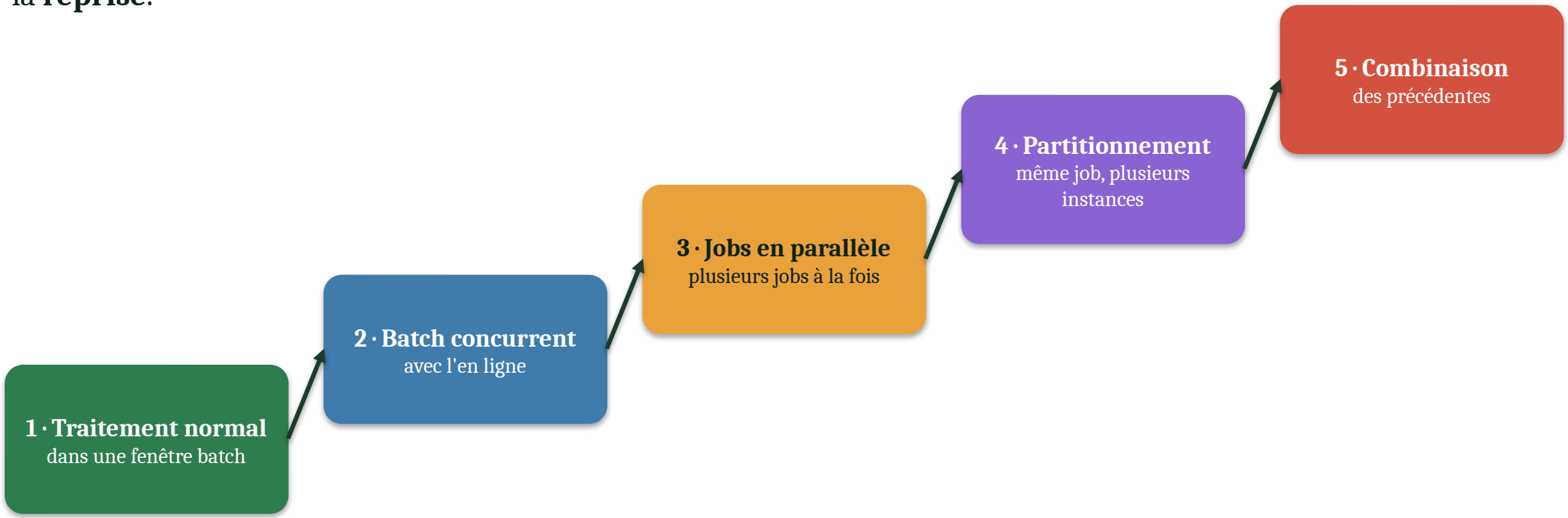
Même les équipes qui n'utilisent pas Spring finissent par l'utiliser pour le batch.

Principes d'un bon batch

Principe	Pourquoi	Ce que ça évite
Minimiser les E/S	premier coût d'un batch	relire à chaque transaction, scans inutiles
Ne pas faire deux fois	un total recalculé relit tout	un second passage complet
Supposer le pire	données non contrôlées = suspectes	un fichier tronqué accepté comme complet
Tester la charge tôt	un batch grossit avec le temps	découvrir la limite en production
Sauvegarder aussi les fichiers	les fichiers plats sont oubliés	un batch qui dépend de fichiers perdus
Rester simple	complexité = risque	des batchs monolithiques ingérables

Stratégies de traitement : du plus simple au plus complexe

Même dans le cas simple : prévoir des **points de validation** et la **reprise**.

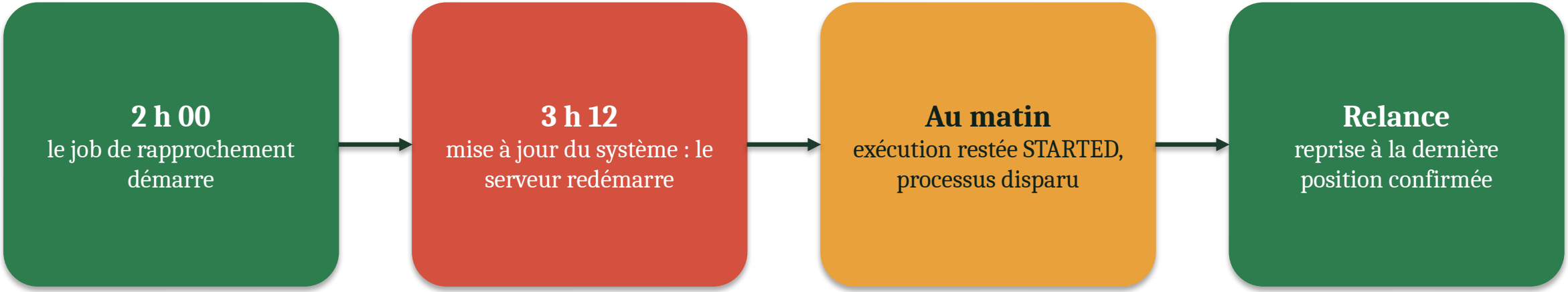


Trois questions de production : trace, verrou, reprise

Question	Ce qui répond
Que s'est-il passé cette nuit, et où le job s'est-il arrêté ?	les **métadonnées** stockées par le `JobRepository`
Deux serveurs peuvent-ils lancer le même job en même temps ?	le **verrouillage** fondé sur ces mêmes métadonnées
Peut-on reprendre sans tout rejouer ni doubler les données ?	l'**état** de chaque step (`ExecutionContext`)

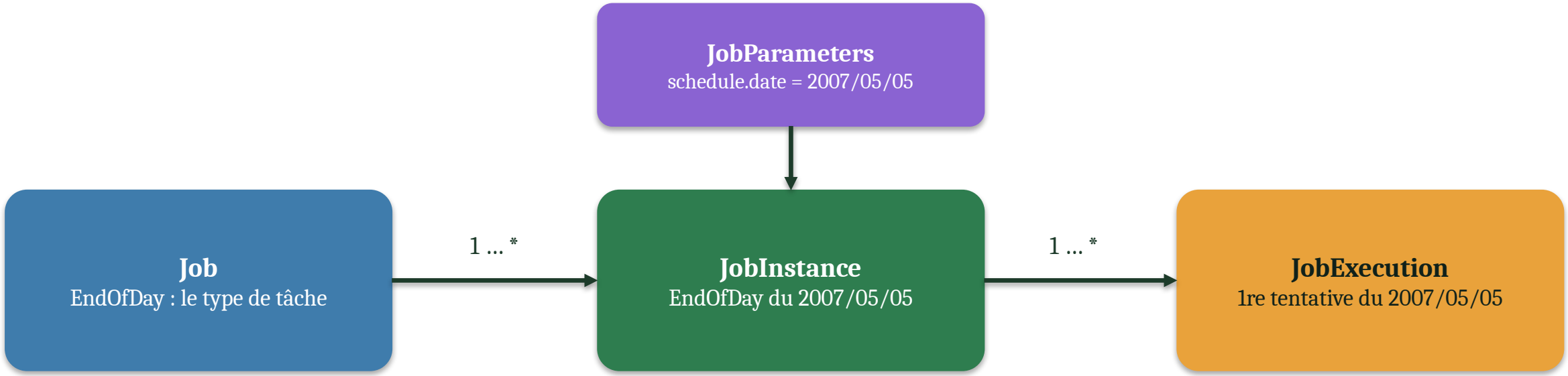
Les trois réponses reposent sur une base **durable et partagée** : PostgreSQL. Une base en mémoire (H2) convient aux tests, jamais à la production.

La nuit interrompue : crash à 3 h 12, reprise au matin



On déclare l'exécution orpheline (`abandon` / `recover`), puis on relance : le step reprend à la **position sauvegardée** au lieu de repartir de zéro.

Job, JobInstance, JobExecution



- **Job** : définit ce qu'est la tâche et comment l'exécuter.
- **JobInstance** : un job + ses paramètres identifiants, une exécution logique.
- **JobExecution** : une tentative réelle, réussie ou échouée.

Schéma officiel à insérer : « Job Hierarchy With Steps » (documentation Spring Batch)

JobInstance = Job + paramètres identifiants

JOB_EXEC_ID	JOB_INST_ID	paramètre schedule.date	STATUS
1	1	2017-01-01	FAILED
2	1	2017-01-01 (relance)	COMPLETED : l'instance 1 est fermée
3	2	2017-01-02	COMPLETED : une AUTRE instance

Mêmes paramètres

Même instance, nouvelle exécution : c'est la **reprise**.

Instance réussie

Fermée : un nouveau lancement identique est **refusé**.

Instance en cours

Si on la relance :

`JobExecutionAlreadyRunningException`

Schéma officiel à insérer : « Job Parameters » (documentation Spring Batch)

Step et StepExecution

Propriété	Rôle
`status` / `exitStatus`	état technique / code de sortie
`readCount` / `writeCount`	éléments lus / écrits
`commitCount` / `rollbackCount`	transactions validées / annulées
`filterCount`	éléments filtrés par le processor
`readSkipCount` / `processSkipCount` / `writeSkipCount`	éléments ignorés (skip)
`executionContext`	données persistées pour la reprise

Si un step échoue parce que le précédent a échoué, aucune StepExecution n'est créée.

ExecutionContext et JobRepository

Contexte du step

Sauvegardé à **chaque commit** : position de lecture, compteurs, vos données.

Contexte du job

Sauvegardé **entre deux steps**. Ce ne sont **pas** les mêmes objets.

Entrées sérialisables

Sans cela, la reprise du job est impossible.

STEP_EXEC_ID	SHORT_CONTEXT
1	`{"bankCsvReader.read.count":3400}`

`BATCH_STEP_EXECUTION_CONTEXT` : le step a lu 3 400 éléments avant l'échec ; la reprise repart de l'élément suivant.

JobRepository : persiste les métadonnées dans **PostgreSQL**. Colonne `version` : verrouillage optimiste (voir 1.5).

Architecture en trois couches

Application
vos jobs, steps, readers, processors

Batch Core
Job, Step, JobOperator, JobRepository

Batch Infrastructure
ItemReader, ItemWriter, retry…

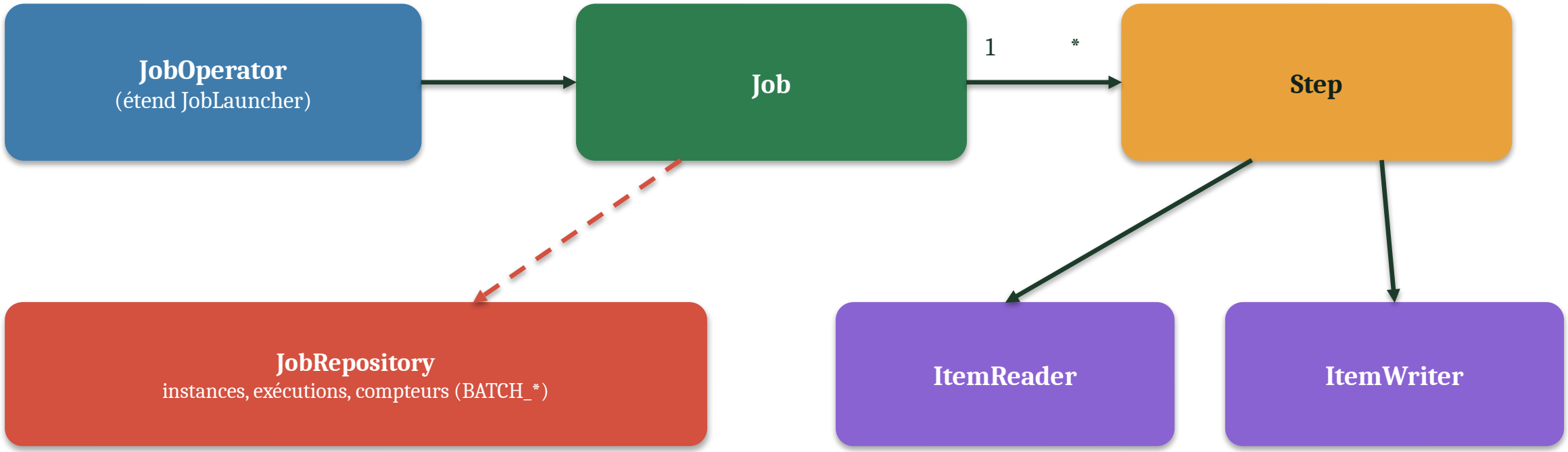
Le code que vous écrivez : les traitements par lots et le code personnalisé.

Le noyau : classes d'exécution pour lancer et contrôler un job.

Base commune : readers, writers et services courants, utilisés par l'application ET par le cœur.

Schéma officiel à insérer : « Spring Batch Layered Architecture » (documentation Spring Batch)

Le schéma de référence : JobOperator, Job, Step, JobRepository



Un step a exactement **un reader**, **un writer**, et un **processor facultatif**.

Historique des versions : de 1.x à 6.x

Version	Période	Apport
1.x	2008	configuration XML
2.x	2009-2010	NoSQL, configuration Java
3.x	Spring 4	Java 8, JSR-352
4.x	Spring 5	alignement sur Spring 5
5.x	Spring 6	Java 17 minimum, Jakarta EE
6.x	Spring 7, Boot 4	`JobOperator`, `ChunkOrientedStep`, retry de Spring Framework 7

Spring Batch N pour Spring Framework N–1. Depuis la v5 : le JobRepository exige une base de données (PostgreSQL en production).

Ce qui change dans Spring Batch 6

Nouveauté (6.0.6)	Conséquence
Boot 4.1.2 embarque Spring Batch 6.0.6	`spring-boot-starter-batch-jdbc`
Paquets déplacés	`core.job.Job`, `core.job.parameters.JobParameters`, `core.step.Step`, `infrastructure.item.*`
`JobOperator` étend `JobLauncher`	`start`, `restart`, `stop`, `abandon`, `recover`
`JobRepository` étend `JobExplorer`	une seule interface pour lire et écrire les métadonnées
`@EnableJdbcJobRepository`	déclare le dépôt JDBC : `dataSourceRef`, `tablePrefix`, `isolationLevelForCreate`…

Dépendances Maven : Boot 4.1.2, Spring Batch 6.0.6, PostgreSQL

```xml
pom.xml (extrait ; groupId omis)
<parent>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.2</version>
</parent>
<properties><java.version>25</java.version></properties>
<dependencies>
  <dependency>
    <artifactId>spring-boot-starter-batch-jdbc</artifactId>
  </dependency>
  <dependency>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <artifactId>spring-boot-starter-batch-jdbc-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

1. Le **parent Boot** fixe toutes les versions compatibles.
2. Compile en **Java 25**.
3. Le starter apporte Spring Batch **et** le support JDBC des métadonnées.
4. Pilote PostgreSQL, uniquement à l'exécution.
5. Outils de test, **jamais** dans l'application livrée.

application.properties : base, lancement automatique, schéma

```
application.properties
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/batchdb}
spring.datasource.username=${DB_USER:batch}
spring.datasource.password=${DB_PASSWORD}
spring.batch.job.enabled=false
spring.batch.jdbc.initialize-schema=never
```

1. `${VARIABLE:défaut}` : l'URL vient de l'**environnement**, pas du dépôt.
2. Un compte **dédié** au batch, sans droits d'administration.
3. **Aucun défaut** pour le mot de passe.
4. Pas de lancement de tous les jobs au démarrage.
5. `never` en production : le schéma vient de **Flyway** ; `always` en développement seulement.

@EnableJdbcJobRepository ligne par ligne

```java
BatchInfraConfig.java
@Configuration
@EnableJdbcJobRepository(
    dataSourceRef = "dataSource",
    transactionManagerRef = "transactionManager",
    tablePrefix = "BATCH_")
public class BatchInfraConfig { }
```

1. Classe de configuration Spring.
2. Déclare un `JobRepository` **JDBC**.
3. Le `DataSource` PostgreSQL de Boot.
4. Le gestionnaire de transactions des métadonnées.
5. Préfixe des tables : plusieurs applications dans un même schéma.

Ne changez pas l'isolation de création

`isolationLevelForCreate` vaut **SERIALIZABLE par défaut**. L'abaisser supprime la protection contre deux lancements simultanés.

Un premier job : tasklet, step, job

```java
FirstJobConfig.java
@Configuration
public class FirstJobConfig {

  @Bean
  Tasklet firstTasklet() {
    return (contribution, chunkContext) -> {
      log.info("Bonjour depuis {}",
          contribution.getStepExecution().getStepName());
      return RepeatStatus.FINISHED;
    };
  }

  @Bean
  Step firstStep(JobRepository repo, PlatformTransactionManager tx,
                 Tasklet firstTasklet) {
    return new StepBuilder("firstStep", repo)
        .tasklet(firstTasklet, tx).build();
  }

  @Bean
  Job firstJob(JobRepository repo, Step firstStep) {
    return new JobBuilder("firstJob", repo).start(firstStep).build();
  }
}
```

1. `Tasklet` : interface fonctionnelle, **une** méthode `execute`.
2. `FINISHED` : le step est terminé (`CONTINUABLE` rappellerait la tasklet).
3. `StepBuilder(nom, repo)` : le nom est la **clé** des exécutions du step.
4. `JobBuilder(nom, repo)` : le nom identifie la recette dans `batch_job_instance`.

Lancer le job par une API : record et switch

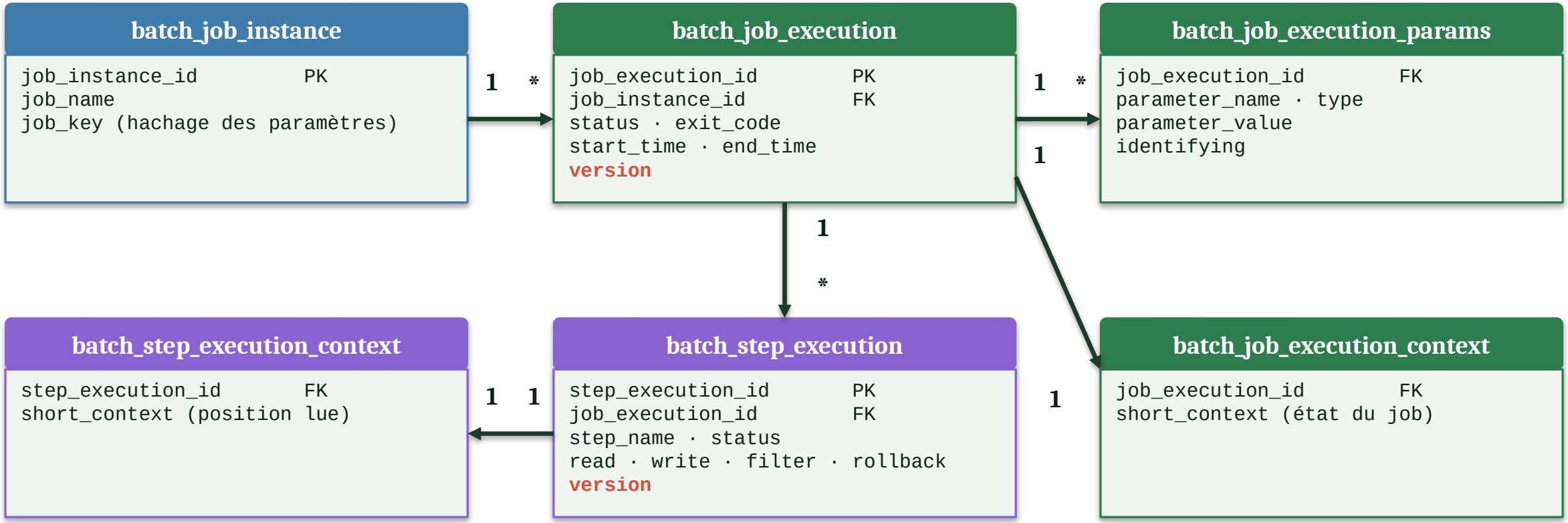
```java
FirstJobController.java (extrait)
public record RunRequest(String runDate) { }

public record RunResult(long executionId, String status) {
  static RunResult of(JobExecution ex) {
    return new RunResult(ex.getId(), switch (ex.getStatus()) {
      case COMPLETED -> "TERMINÉ";
      case FAILED, STOPPED -> "À REPRENDRE";
      default -> ex.getStatus().name();
    });
  }
}

@PostMapping("/jobs/first")
RunResult run(@RequestBody RunRequest req) throws Exception {
  var params = new JobParametersBuilder()
      .addString("runDate", req.runDate())
      .toJobParameters();
  return RunResult.of(jobOperator.start(firstJob, params));
}
```

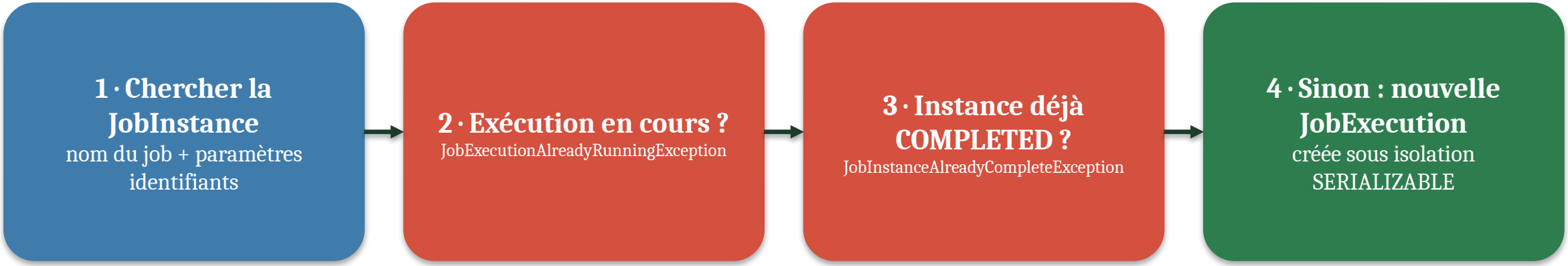
1. `record` : données **immuables**, sans code répétitif.
2. `switch` en **expression** sur `BatchStatus`.
3. **POST**, jamais GET : un robot ne doit pas relancer le job.
4. `runDate` est **identifiant** : il définit la `JobInstance`.
5. `JobOperator.start` lance et **attend** la fin.

Ce que le JobRepository écrit : 6 tables, 3 séquences



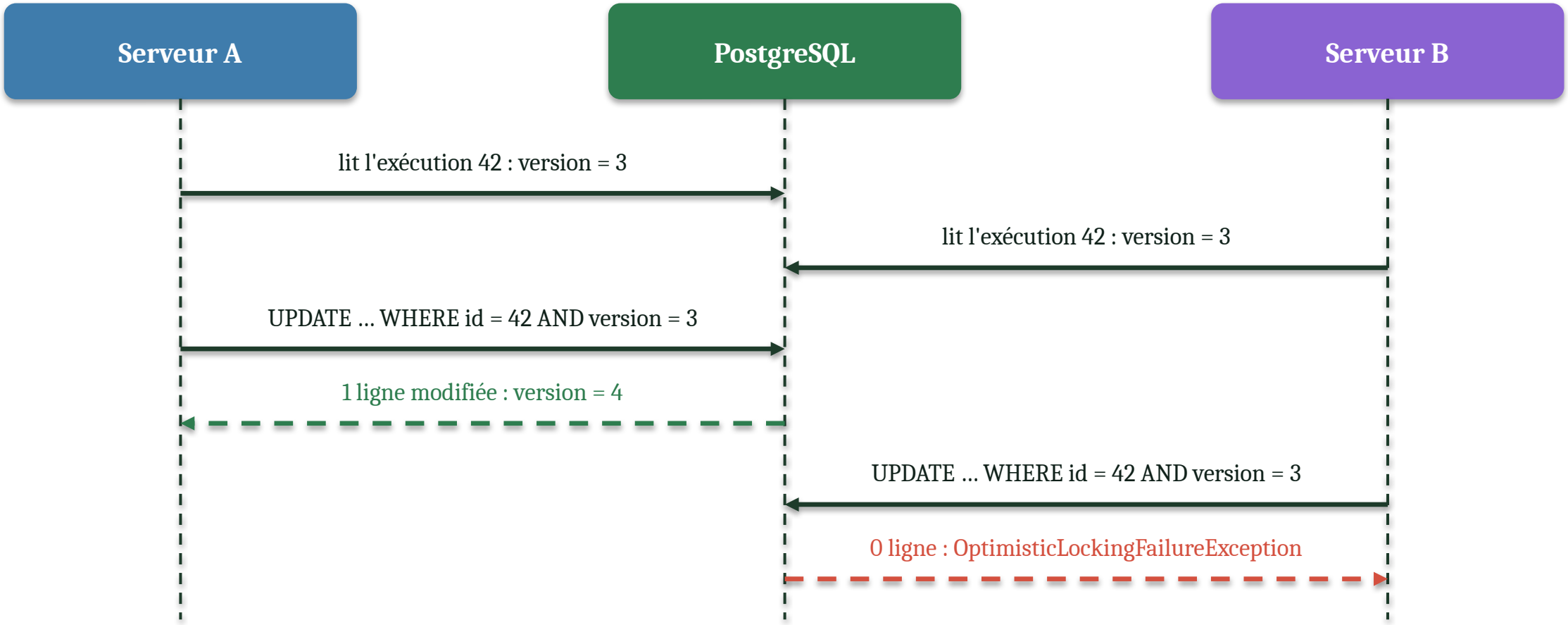
3 séquences : `batch_job_instance_seq`, `batch_job_execution_seq`, `batch_step_execution_seq`. Colonnes `version` en rouge : verrouillage optimiste.

Comment un lancement est vérifié



Le perdant d'une course reçoit une exception : le lancement doit être **retenté** ou **abandonné**.

Le verrouillage optimiste, pas à pas



Le second serveur écrit avec une **version périmée** : aucune ligne ne correspond, l'écriture est refusée.

Ce qu'il faut savoir sur le verrouillage optimiste

Le mécanisme

Colonne `version` sur chaque ligne.

```
UPDATE … WHERE id = ? AND version = ?
```

Aucune ligne touchée : `OptimisticLockingFailureException`.

Le piège

En théorie, une copie périmée lève l'exception.

En pratique, `JdbcJobExecutionDao` **resynchronise la version** avant l'écriture pour ne pas écraser un ordre d'arrêt.

Le verrouillage existe, mais il est **atténué**.

À retenir

Ne vous attendez pas à une exception à chaque fois.

Deux lancements simultanés : exécutions en cours + SERIALIZABLE.

Arrêt depuis l'extérieur : `JobOperator.stop(...)`.

À retenir : c'est l'isolation SERIALIZABLE, pas le numéro de version, qui arrête deux lancements simultanés du même job.

Pourquoi le JobRepository est un singleton sans danger

Singleton sans état : sûr

Le `JobRepository` ne conserve **rien** entre deux appels : tout l'état vit **en base**.

Exemples : `JobRepository`, `JobOperator`, un `DataSource`.

Singleton avec état : piège

Un total ou un compteur en mémoire se partage entre **tous** les lancements : c'est le **double comptage** du Module 4.

Exemple : un total de 135,50 qui devient 271,00 puis 406,50.

Une seule instance pour toute l'application : sans danger quand elle ne retient rien.

Deux serveurs, un seul traitement ; l'audit

Cas	Ce qui répond
Deux serveurs lancent le **même job** à 2 h 00	une exécution démarre ; l'autre échoue proprement (`JobExecutionAlreadyRunningException`)
« Quel a été le résultat du job d'hier ? »	une requête SQL, sans rien ajouter à l'application

```
Audit : compteurs d'un step
SELECT step_name, status, read_count, write_count,
       filter_count, skip_count
  FROM batch_step_execution
 WHERE job_execution_id = ?;
```

Top 10 des pièges (1/2)

#	Piège	Ce qui se passe	Parade
1	`initialize-schema=always` en production	l'application crée des tables au démarrage ; droits excessifs	`never` + Flyway / Liquibase
2	`JobRepository` en mémoire (H2) en production	aucune reprise après redémarrage, aucune trace	PostgreSQL persistant
3	`isolationLevelForCreate` abaissé	deux lancements concurrents possibles	garder SERIALIZABLE, **gérer** l'erreur
4	Relancer avec les mêmes paramètres	`JobInstanceAlreadyCompleteException`	un paramètre identifiant qui change, ou `startNextInstance`
5	Horodatage en paramètre identifiant	chaque relance est une nouvelle instance : plus de reprise	paramètre métier stable (`runDate`)

Top 10 des pièges (2/2)

#	Piège	Ce qui se passe	Parade
6	`spring.batch.job.enabled` laissé à `true`	tous les jobs partent au démarrage	`false` + lancement explicite
7	Exécution restée `STARTED` après un crash	le job refuse de repartir	`abandon` / `recover` après vérification
8	Tables `BATCH_*` jamais purgées	tables énormes, requêtes lentes	purge planifiée
9	Imports de Spring Batch 4/5	erreurs de compilation	utiliser les paquets 6.x
10	Job long dans la requête HTTP	requête bloquée, délai dépassé	lancement asynchrone (Module 2)

Exercices du Module 1

1. **Même paramètre.** Vous lancez deux fois `firstJob` avec `runDate=2026-11-01`. Que se passe-t-il au second lancement si le premier est `COMPLETED` ? Et s'il est encore en cours ? Citez les deux exceptions.
2. **Paramètre de relance.** Relancer le job du jour autant de fois que nécessaire après un échec, sans jamais rejouer un job réussi : quel paramètre est identifiant, lequel est non identifiant ? Justifiez.
3. **Isolation.** Un collègue propose `READ_COMMITTED` pour « supprimer les erreurs de sérialisation ». Quel risque cela crée-t-il, et que lui répondez-vous ?